

```
    echo $2
}
digit_func 'First_Argument'
digit_func 'First_Argument' 'Second Argument'
```

produces the output:

```
Hey! We are inside a function.
First_Argument
```

```
Hey! We are inside a function.
```

```
First_Argument
```

```
Second Argument
```

since, there is no second argument in the first call, \$2 is blank, and then `echo $2` command merely works a newline.

Dealing with command line parameters

It is done in much the same way as arguments to a function. For example on issuing the following command

```
$ ./script_name.sh param1 param2 param3
```

`param1` would be assigned variable name `$1`, `param2` as `$2` and `param3` as `$3` and here is how we can deal with them. We would make a simple script that would delete a single file, which can be passed on as a command line argument. `rm` command standing for remove is used in Linux for deleting the file, if you find it difficult to remember, create a script called `./delete`. `sh` and store it with the following contents:

```
#!/bin/bash
# Declaring a function
echo 'deleting file $1'
echo `rm $1`
echo 'deleted the file $1'
```

and now if you execute the following in the shell:

```
$ ./delete.sh file_to_be_deleted.txt
it will just do the job.
```

User-Input, Bash Trap and If-Else

Till now we haven't considered any example where we had to take input